

Jett Architecture & File Guide

Copyright © 2026 Shashwat Ghadge. All rights reserved.

Pipeline: .jett source → Lexer → Tokens → Parser → AST → Type Checker → Interpreter → output.

Repository:

- .github/workflows/build.yml: CI build and CTest on Ubuntu and Windows.
- CMakeLists.txt: C++17 build configuration, warnings, and CTest registrations.
- README.md: public overview, syntax examples, build/run instructions, and architecture summary.
- LICENSE: proprietary usage restrictions.
- COPYRIGHT.md: copyright and branding ownership notice.
- assets/: Jett logos, wordmark, banner, tagline, and .jett icon.
- examples/arrays.jett: array creation/indexing example.
- examples/calculator.jett: arithmetic example.
- examples/functions.jett: functions and returns example.
- examples/hello.jett: introductory syntax example.
- examples/input.jett: input example.
- examples/loops.jett: loop example.
- tests/invalid_type.jett: intentional type-check failure.
- tests/invalid_syntax.jett: intentional parser failure.
- src/token.h: token types and Token structure.
- src/token.cpp: token type names for diagnostics.
- src/lexer.h: lexer interface and scanning helpers.
- src/lexer.cpp: lexical analysis, comments, literals, operators, keywords, and positions.
- src/ast.h: AST node types for expressions, statements, functions, arrays, and Program.
- src/ast.cpp: AST translation unit; most structures are inline in ast.h.
- src/ast_printer.h/.cpp: AST debugging printer.
- src/parser.h/.cpp: recursive-descent parser that creates the AST.
- src/type_checker.h/.cpp: static type validation and type errors.
- src/interpreter.h/.cpp: runtime values, execution, operators, arrays, functions, control flow, and built-ins.
- src/main.cpp: CLI entry point and full source-to-execution pipeline.
- vscode-jett/package.json: VS Code extension metadata and language contributions.
- vscode-jett/language-configuration.json: comments, brackets, auto-closing pairs.
- vscode-jett/syntaxes/jett.tmLanguage.json: Jett syntax highlighting.
- vscode-jett/jett-icon-theme.json: .jett file icon mapping.
- vscode-jett/icons/: extension icon assets.
- windows/: Windows-specific support.
- .vscode/: development/editor configuration.
- .gitignore: ignored build/editor artifacts.

How execution works

1. main.cpp reads a .jett file and handles --help/--version.
2. Lexer converts characters into Tokens with line/column positions.
3. Parser consumes Tokens using recursive descent and constructs Program/AST nodes.
4. TypeChecker validates declarations, expressions, arrays, indexing, calls, and assignments.
5. Interpreter evaluates the AST using std::variant runtime values, shared array storage, function tables, and control-flow signals.

Development validation

Configure with CMake, build the jett executable, then run ctest --output-on-failure. GitHub Actions repeats the build/test process on Ubuntu and Windows.